

CWM WORK JOURNAL 4

Q1:

I use GPT to help me explain the function of each layer and the rules of assigning addresses.

Item	What it holds	How its address is assigned	
<code>.text</code>	Machine instructions	Fixed by the linker/loader as part of the program image; on Linux/x86-64 it is typically mapped near a low virtual address, but exact placement can vary with ASLR.	
<code>.data</code>	Initialized global and static variables	Placed by the linker after <code>.text</code> ; the loader maps it into memory at runtime.	
<code>.bss</code>	Uninitialized global and static variables, zero-filled at load time	Placed after <code>.data</code> ; it gets an address from the linker, then the loader allocates zeroed memory for it.	
heap	Dynamic allocations from <code>malloc</code> , <code>new</code> , etc.	Not fixed at link time; the OS grows it at runtime as the program requests memory.	
stack	Function call frames, local variables, saved registers, return addresses	Not fixed at link time; the OS sets up the initial stack near high virtual addresses, and each function call moves the stack pointer.	

Q2&3:

```
ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ ./main 8
Target Function Ptr: 0x4011dc
ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ ./main 24
Target Function Ptr: 0x4011dc
Success! Malicious function Called!
Segmentation fault (core dumped)
ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ ./main 30
Target Function Ptr: 0x4011dc
Segmentation fault (core dumped)
ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ ./main 40
Target Function Ptr: 0x4011dc
Segmentation fault (core dumped)
ubuntu@ubuntu:~/CWM-FDI/assignment4/buffer_overflow/4A$ ./main 60
Target Function Ptr: 0x4011dc
Segmentation fault (core dumped)
```

From the results, we can see that for all input addresses, it returns the target function that the control follows originally intended to use. However, for large inputs, the output includes “Segmentation fault” as the return address is overwritten, meaning some parts of the addresses are invalid. For input offset 24, the return includes the malicious function.

Q4:

```

'''
Higher addresses (toward top of address space)
+-----+
|   return address   | <-- where to go when the function returns
+-----+
|   saved RBP        | <-- caller's base pointer
+-----+
|  local variables   |
+-----+
|   buf[16]          | <-- arrays live here
+-----+
Writing to buf moves stack pointer UPWARDS
'''

```

In this case, the offset 24 lands just at the edge of the return address and saved RBP, as the total bytes of saved RBP, local variables, and buffer is 24 bytes. However, any number larger than 24 will overwrite the function and return a useless return address.

Q5:

When I input an offset less than 12, the output only returns the address, which implies that there is a guaranteed returning function output. However, when I use a large offset like over 100, the output will include “segmentation fault”, which implies that we can manipulate the returning output of the function here.

Q6:

The order of bytes outputs is reversed compared to 0xdeafbeef originally written in hex.

Buf Offset	Address	Content (hex)
Buf[0]	0xef	ef
Buf[1]	0xbe	be
Buf[2]	0xad	ad
Buf[3]	0xde	de

Q7:

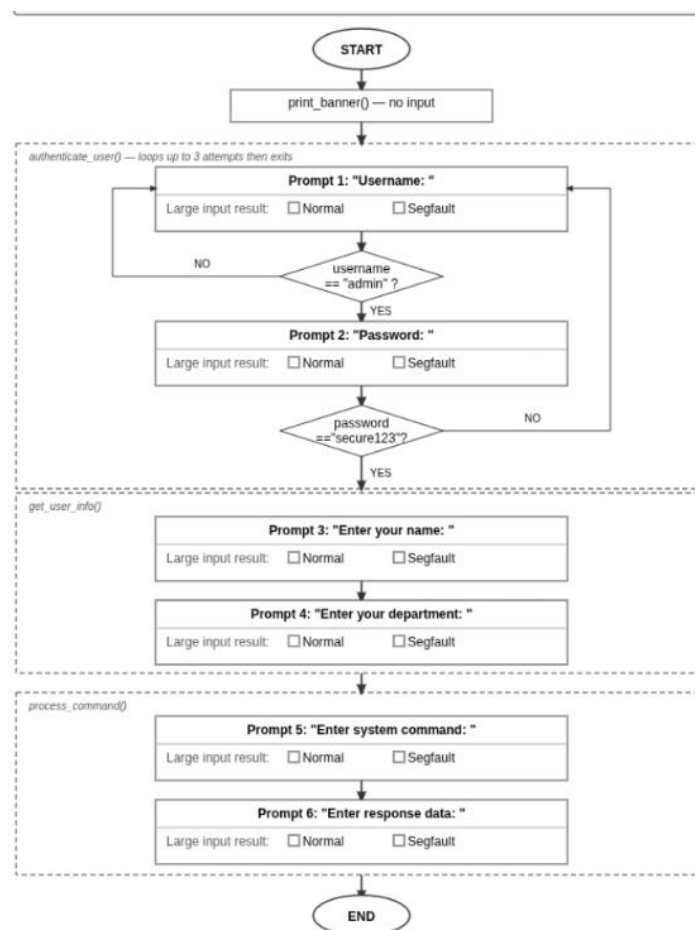
Strcpy is a culprit here as the function does not check the input size of the string. In this case, the function is easy to overwrite the buffer range and rewrite the saved RBP and

return address, which makes attack possible. An available replacement would be `strncpy()`, where this function limits the bytes copied and prevents from exceeding the limit of buffer.

Q8&9:

The program includes an authentication system, in which I should input the correct username and password to login to the system.

Q10:



From test results,

1. prompt 1 "username" is normal.
2. Prompt 2 "password" is normal.
3. Prompt 3 to 6 has segmentation fault, which shows that this part of the system is vulnerable.

0	0
1	0
2	0
3	0
4	0
5	2
6	0
7	0
8	0
9	0

Q21:

Guessing 5 gives a return time much longer than other numbers, telling me that the correct first character is 5.

Guess	Time
50	2.1
51	4.3
52	2.1
53	2.2
54	2.2
55	2.1
56	2.3
57	2.2
58	2.2
59	2.1

Q25:

The attack.py returns 519265.

```
[+] prefix so far: '519265'
[*] Password cracked: '519265'
```

The captured flag is:

```
Access granted. Flag: CWM{t1m1ng_1s_3v3ryth1ng}
Check took: 12.8 ms
```

Q26:

Deleting the `time.sleep` function in the loop will remove the huge time gap. However, fundamentally, the code enumerates the input password and compares it letter by letter with the correct one, so there will still be a minute time difference produced for different numbers of correct characters.

Q27:

To be honest, I do not realize that there's so many functions that are functionally correct but underlying with so many security problems. I think I will be much more cautious about using API in a secure-critical environment. As before, I only focus on the functionality of my code but never pay any attention to the security of my code. For third party APIs, if my code is not secured, there's a large possibility of leaking my information.